

Git Merge vs Git Rebase - Complete Interview Guide

1. Git Merge

Git merge combines two branches and creates a merge commit. It preserves history.

```
git checkout main
git merge feature-branch
```

Merge Diagram

```
A---B---C (main)
      \
      D---E (feature)

After:
A---B---C-----M
      \   /
      D---E
```

CMD View (Merge)

```
$ git log --oneline --graph --all
*    M (merge commit)
| \
|  * E
|  * D
*  | C
*  | B
*  | A
```

2. Git Rebase

Git rebase moves commits on top of another branch and rewrites history.

```
git checkout feature-branch
git rebase main
```

Rebase Diagram

```
A---B---C (main)
      \
      D---E (feature)

After:
A---B---C---D'---E'
```

CMD View (Rebase)

```
$ git log --oneline --graph --all
* E'
```

* D'
* C
* B
* A

3. Difference Between Merge and Rebase

Point	Git Merge	Git Rebase
History	Keeps original history (non-linear)	Rewrites history (linear)
Commit	Creates merge commit	No merge commit
Structure	Branch structure visible	Straight line history
Safety	Safe for shared branches	Risky if already pushed
Conflicts	Resolve once	Resolve multiple times
Usage	Team collaboration	Clean history before push
Visibility	Shows full timeline	Hides branch details

4. Merge Conflict

Occurs when same file and same line is modified differently.

```
<<<<<<< HEAD
Main code
=====
Feature code
>>>>>>> feature-branch
```

Steps to Resolve Conflict

1. Open file 2. Remove markers 3. Keep correct code 4. git add file.txt 5. git commit

5. Rebase Conflict

```
git rebase main
# fix conflict
git add file.txt
git rebase --continue
```

6. View Diagram in CMD

```
git log --oneline --graph --all
```

7. Summary

Merge = Combine history (safe, non-linear) Rebase = Rewrite history (clean, linear) git log --graph
= Visualize diagram